

Kapitel 1 · Eingaben und Variablen

Fahrradverleih · Einstieg in Python

ORIENTIERUNG Blau: Lernmaterial · Gelb: Merksätze · Grün: Arbeitsaufträge · Grau: Code

Das lernen Sie

Sie können ein Python-Programm in einer IDE speichern und starten, Eingaben in Variablen ablegen und Zahlen aus Text erzeugen. Damit bereiten Sie die Daten für den Datenbankeintrag eines neues Fahrrads vor.

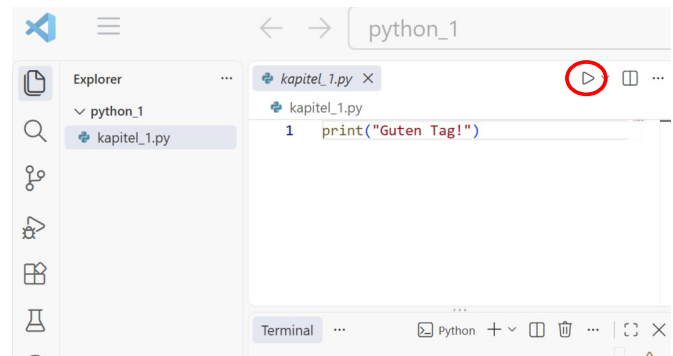
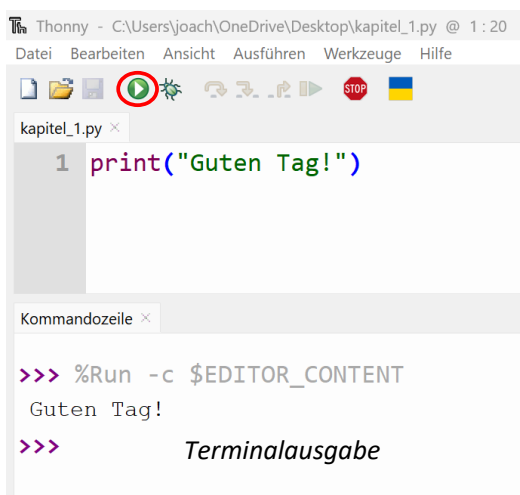
1. Arbeiten in einer Python-IDE

Allgemeine Grundlagen: Quellcode und Ausführung

Ein Python-Skript ist eine Textdatei mit der Endung .py. Darin stehen Anweisungen in der Programmiersprache Python. Der Editor dient zum Schreiben; der Python-Interpreter führt das Programm aus. Die IDE verbindet diese Werkzeuge in einer Oberfläche. In der Konsole erscheinen Ausgaben und dort beantworten Sie Eingabeaufforderungen.

Thonny (Integrated Development and Learning Environment)

IDE Visual Studio Code (VSC)



Terminalausgabe

 Ausführen Quellcode (Aufruf Python-Interpreter)

In den einfachen Programmen dieses Kapitels werden die Anweisungen nacheinander von oben nach unten ausgeführt. Python erwartet eine bestimmte Schreibweise (Syntax). Fehlende Anführungszeichen oder Klammern können einen SyntaxError auslösen. Lesen Sie bei einer Fehlermeldung zuerst den Fehlertyp und die angegebene Zeile. Prüfen Sie auch die Zeile davor. Kommentare mit # erläutern den Code.

Allgemeines Beispiel: Speichern Sie die folgenden Zeilen als begruessung.py und starten Sie die Datei. Die beiden Texte erscheinen in dieser Reihenfolge.

```
print("Guten Tag!")
print("Hallo") # Gibt eine Begrüßung aus.
```

Anwendung: Fahrradverleih

Legen Sie eine neue Datei an, speichern Sie sie als uebung_01.py.

Schreiben Sie zunächst diese Zeile und starten Sie das Programm:

```
print("Fahrrad erfassen")
```

Text steht in Anführungszeichen. Die Klammern gehören zum Funktionsaufruf. Schreiben Sie jede Anweisung auf eine eigene Zeile.

2. Variablen und Datentypen

Allgemeine Grundlagen: Namen, Werte und Typen

Variablen verweisen auf Werte, die während eines Programmlaufs verwendet werden.

Damit mit den Daten im Programm weiter gearbeitet werden kann, werden sie in Speicherzellen gespeichert. Damit die Daten unterscheidbar sind, müssen die Speicherzellen Adressen (Namen) erhalten.

Variablenamen dürfen Buchstaben, Ziffern und Unterstriche enthalten, aber nicht mit einer Ziffer beginnen. Leerzeichen sind nicht erlaubt. Aussagekräftige Namen wie `anzahl` sind leichter zu verstehen als `x`.

Der Datentyp gehört zum Wert: `str` steht für Text, `int` für ganze Zahlen, `float` für Gleitkommazahlen und `bool` für Wahrheitswerte. Python erkennt den Typ anhand des Wertes. Ein Text steht im Quellcode in Anführungszeichen. Zahlen und die Wahrheitswerte `True` und `False` stehen ohne Anführungszeichen.

Beispiel: `anzahl = 3` speichert eine ganze Zahl; `anzahl = "3"` speichert einen Text. Die Zeichen sehen ähnlich aus, haben aber unterschiedliche Bedeutung. `float` speichert viele Dezimalzahlen nur näherungsweise.

Merksatz – Variablen: Ein Name verweist auf einen Wert. Eine neue Zuweisung ersetzt die bisherige Zuordnung.

```
anzahl = 3
anzahl = 5
print(anzahl) # Ausgabe: 5
```

Merksatz – Namen: Verwenden Sie verständliche Namen ohne Leerzeichen; eine Ziffer darf nicht am Anfang stehen. Beispiel: `anzahl_personen` ist zulässig, `2personen` nicht.

Merksatz – Datentypen: Anführungszeichen kennzeichnen Text. Ganze Zahlen, Gleitkommazahlen und Wahrheitswerte haben eigene Datentypen.

ganze Zahlen:

```
>>> a=1
>>> b=2
>>> a+b
3
>>> type(a)
<class 'int'>
```

Binäre Speicherung der Zahl 1

a	0	0	0	0	0	0	0	1
b	0	0	0	0	0	0	1	0
d	0	1	0	0	0	0	1	1

der Inhalt der Variable d wird nicht als Zahl, sondern als ascii-Code für C interpretiert → Datentyp String (str)

Zeichenketten:

```
>>> c='Abend'
>>> d='C'
>>> c+d
'AbendC'
>>> type(c)
<class 'str'>
```

Anwendung: Fahrradverleih

Eine Variable ist ein Name für einen Wert. Mit `=` weisen Sie ihr einen Wert zu. Python unterscheidet Groß- und Kleinschreibung: `artikelNr` und `artikelnr` sind verschiedene Namen. Verwenden Sie die Namen aus dem Beispiel durchgängig.

<code>rahmenNr = "AB1234"</code>	<code># str: Text</code>
<code>artikelNr = 4711</code>	<code># int: ganze Zahl</code>
<code>tagesmietpreis = 12.5</code>	<code># float: Kommazahl</code>
<code>eingabe_ok = True</code>	<code># bool: Wahrheitswert</code>

`str` eignet sich für die Rahmennummer: Sie kann Buchstaben und führende Nullen enthalten. `int` eignet sich für Artikelnummer, Jahr, Monat und Tag. `float` wird in der Vorlage für den Tagesmietpreis verwendet. `bool` hat die Werte `True` und `False`; Sie brauchen Wahrheitswerte später bei Bedingungen.

3. Benutzereingabe mit input()

Allgemeine Grundlagen: Aufruf und Rückgabewert

Eine Funktion ist ein wiederverwendbarer Codeblock, der eine bestimmte Aufgabe erledigt und erst bei seinem Aufruf ausgeführt wird. Angaben innerhalb der Klammern heißen Argumente. Manche Funktionen liefern ein Ergebnis zurück: den Rückgabewert. Mit einer Zuweisung können Sie dieses Ergebnis in einer Variablen aufbewahren.

Bei `input("Name: ")` ist "Name: " der Aufforderungstext. Die eigentliche Antwort tippen Sie beim laufenden Programm in die Konsole und bestätigen sie mit Enter. `input()` liefert den eingegebenen Text ohne den abschließenden Zeilenumbruch.

Merksatz – Funktionen: Argumente gehen in einen Funktionsaufruf hinein; ein Rückgabewert kann als Ergebnis herauskommen und einer Variablen zugewiesen werden.

Merksatz – `input()`: Auch eine Eingabe aus Ziffern wird zunächst als Text zurückgegeben.

```
antwort = input("Anzahl: ")
print(antwort)
```

Beispiel: Bei der Eingabe 12 ist der Rückgabewert der Text "12". "Anzahl: " ist das Argument von `input()`; `antwort` speichert dessen Rückgabewert. Die Ausgabe zeigt 12, beweist aber noch keinen Zahlentyp.

Allgemeine Grundlagen: Parsen und Typumwandlung

Parsen bedeutet, Text nach vorgegebenen Regeln zu lesen und seine Bedeutung zu ermitteln. Hier werden Zahlentexte mit `int()` oder `float()` in Zahlen umgewandelt. Dabei entsteht ein neuer Wert. Der Text "25" und die Zahl 25 sind nicht derselbe Datentyp. Eine Umwandlung ist nötig, wenn Sie eine Eingabe anschließend als Zahl verarbeiten möchten.

In `int(input("Anzahl: "))` wird zuerst `input()` ausgeführt; anschließend bekommt `int()` dessen Rückgabewert.

Merksatz – Parsen: Erst eine passende Umwandlung macht aus einem Zahlentext einen Zahlenwert: `int()` für Ganzzahltext, `float()` für Dezimaltext mit Punkt.

```
anzahl_text = "12"
anzahl = int(anzahl_text) # Zahl 12
preis = float("2.50") # Zahl 2.5
```

Beispiel: `anzahl_text` bleibt ein `str`, während `anzahl` ein `int` ist. `float("2,50")` kann den Text mit Komma hier nicht lesen und führt zu `ValueError`.

Merksatz – Verschachtelte Aufrufe: Der innere Aufruf liefert den Wert für den äußeren Aufruf.

Anwendung: Fahrradverleih

`input()` zeigt einen Aufforderungstext, wartet auf die Eingabe und liefert nach Enter immer einen `str` – auch bei Ziffern.

```
rahmenNr = input("Rahmennummer: ")
artikel_eingabe = input("Artikelnummer: ")
artikelNr = int(artikel_eingabe)
tagesmietpreis = float(input("Tagesmietpreis mit Punkt: "))
jahr = int(input("Anschaffungsjahr: "))
```

Noch wichtig: `float()` erwartet hier einen Dezimalpunkt. `int()` erwartet einen ganzzahligen Text, beispielsweise "2026". Ungültige Eingaben führen zu `ValueError` und beenden das Programm. In dieser Übung verwenden wir gültige Zahlen; die Behandlung beliebiger Fehleingaben ist kein Thema dieser Reihe.

4. Ausgabe mit print()

Allgemeine Grundlagen: Werte sichtbar machen

print() schreibt eine lesbare Darstellung seiner Argumente in die Konsole. Text in Anführungszeichen wird wörtlich ausgegeben; bei einem Variablennamen ohne Anführungszeichen wird der zugeordnete Wert ausgegeben. Beispiel: Nach `anzahl = 3` zeigt `print("anzahl")` das Wort `anzahl`, während `print(anzahl)` die Zahl 3 zeigt.

Mehrere Argumente werden standardmäßig mit einem Leerzeichen getrennt. Nach jedem print()-Aufruf beginnt die nächste Ausgabe in einer neuen Zeile. print() ohne Argument erzeugt eine Leerzeile. Sie können Texte und Zahlen zusammen ausgeben, ohne die Zahl vorher selbst in Text umzuwandeln.

Merksatz – print(): Text in Anführungszeichen erscheint wörtlich; ein Variablenname ohne Anführungszeichen liefert seinen Wert für die Ausgabe.

```
anzahl = 3
print("anzahl") # Ausgabe: anzahl
print(anzahl) # Ausgabe: 3
print("Anzahl:", anzahl) # Ausgabe: Anzahl: 3
```

Merksatz – Mehrere Ausgaben: print() trennt mehrere Argumente standardmäßig durch Leerzeichen und beendet seine Ausgabe mit einem Zeilenumbruch. Eine Ausgabe speichert noch keine Daten dauerhaft.

Anwendung: Fahrradverleih

print() kann mehrere Werte anzeigen. Trennen Sie sie im Aufruf durch Kommas; Python setzt bei der Ausgabe automatisch Leerzeichen dazwischen.

```
print("Rahmennummer:", rahmenNr)
print("Tagesmietpreis:", tagesmietpreis, "Euro")
```

Die Ausgabe 12.5 ist als Zahl genauso richtig wie 12.50. Eine besondere Preisformatierung brauchen Sie für das Speichern in der Datenbank nicht.

Bezug zur Gesamtaufgabe

Das spätere Datenbankprogramm benötigt vier Angaben: Rahmennummer, Tagesmietpreis, Anschaffungsdatum und Artikelnummer. Das Datum erfassen wir zunächst als drei ganze Zahlen. In Kapitel 2 entsteht daraus ein Datumswert.

Kapitel 1 · Pflichtübung

Auftrag: Einen Fahrrad-Datensatz vorbereiten

Erstellen Sie `uebung_01.py`. Verwenden Sie nur Variablen, `input()`, `int()`, `float()` und `print()`; Imports und Datenbankzugriff brauchen Sie noch nicht.

1. Geben Sie zu Beginn „Fahrrad erfassen“ aus.
2. Fragen Sie nacheinander Rahmennummer, Tagesmietpreis, Anschaffungsjahr, Anschaffungsmonat, Anschaffungstag und Artikelnummer ab.
3. Speichern Sie die Rahmennummer als Text, den Preis als float und die übrigen Angaben als int. Verwenden Sie `rahmenNr`, `tagesmietpreis`, `jahr`, `monat`, `tag` und `artikelNr` als Variablennamen.
4. Geben Sie alle sechs Werte mit verständlichen Beschriftungen aus. Geben Sie Jahr, Monat und Tag in einer gemeinsamen Zeile aus.

Probelauf

Geben Sie nacheinander AB1234, 12.50, 2026, 9, 8 und 4711 ein. Die Zusammenfassung soll Folgendes enthalten:

```
Rahmennummer: AB1234
Tagesmietpreis: 12.5 Euro
Anschaffung (Jahr Monat Tag): 2026 9 8
Artikelnummer: 4711
```

Testen Sie danach die Rahmennummer 001234: Die führenden Nullen müssen erhalten bleiben.

Erledigt, wenn ...

Das Programm lässt sich aus der IDE starten; alle Angaben werden abgefragt und richtig ausgegeben. Sie können erklären, warum `input()` allein keine Zahl liefert und warum die Rahmennummer ein `str` bleibt.

Selbstkontrolle

Welcher Aufruf wandelt "2026" in eine ganze Zahl um? Warum verursacht `float("12,50")` hier einen Fehler? Beantworten Sie dies zunächst selbst und vergleichen Sie dann mit den Erläuterungen in Abschnitt 3.

Zum Nachlesen

Python-Tutorial, Ein- und Ausgabe: <https://docs.python.org/3/tutorial/inputoutput.html>

Python-Funktionen `input()`, `int()`, `float()`, `print()`: <https://docs.python.org/3/library/functions.html>